```
% Copyright (C) 2019 Mette S. Olufsen.
% All Rights Reserved.
%
% Note: This a trial version built to develop and test a research version.
%
% Permission is hereby granted, free of charge, to any person obtaining a copy of
% this software and associated documentation files (the "Software") for research and
% development purposes subject to the following conditions:
%
% The above copyright notice and the README.txt file shall be included in all copies of
% any portion of this data.
%
% Whenever reasonable and possible in publications and presentations when this model or
% data is used in whole or part, please include an acknowledgement to the
% manuscript listed below and cite the official website for this code: https://wp.math.ncsu.edu/cdg/
% The software can befound under supplemental material
%
% Explicit written permission should be obtained from M.S. Olufsen prior to posting or
% distributing any portion of this software. Please email msolufse@ncsu.edu to obtain
% written permission.
%
% THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
% IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS
% FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR
% COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER
% IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
% CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

function  DriverBasic(PID,opt)
% Solves the five compartment model described in detail in the manuscript
% by Williams, Brady, Gilmore, Gremaud, Tran, Ottesen, Mehlsen and Olufsen,
% in J Math Biol, Cardiovascular dynamics during head-up tilt assessed via
%pulsatile and non-pulsatilemodels, https://doi.org/10.1007/s00285-019-01386-9
close all;
global ODE_TOL % absolute and relative tolerance for ODE solver


% [mdata,data] = LoadData(datn);

%Load nominal parameter values (pars) and initial conditions for the ODEs
[pars parsG Init] = load_global(PID);
% [pars parsG Init] = load_global(PID, data)
pars = exp(pars);

% Load optimized parameters if opt = 1
% if opt == 1
%   s = strcat('../Optimization/Opt',num2str(datn),'.mat');
```

```
48 %   load(s);
49 %   pars = exp(parsLM);
50 % end
51
52 % Display parameter values
53 % if opt == 1
54 %     disp('Estimated parameters')
55 % else
56 %     disp('Nominal parameters')
57 % end
58
59 pars
60
61 % Define resistances
62 Rpul  = pars(1);
63 Rmval = pars(2);
64 Raval = pars(3);
65 Rsys   = pars(4);
66 Rtval  = pars(5);
67 Rpval  = pars(6);
68
69 % Define compliances
70 Cpv = pars(7);
71 Csa    = pars(8);
72 Csv    = pars(9);
73 Cpa = pars(10);
74
75 % Heart parameters
76 ElvM = pars(11);
77 Elvm = pars(12);
78 ErvM = pars(13);
79 Ervm = pars(14);
80
81 TC = pars(15);
82 TR = pars(16);
83
84 %Initialize empty vectors for solutions for pressure
85 PpvSa  = [];
86 PlvSa  = [];
87 PsaSa  = [];
88 PsvSa  = [];
89 PrvSa  = [];
90 PpaSa  = [];
91
92 %Initialize empty vectors for solutions for volume
93 VpvSa  = [];
94 VlvSa  = [];
95 VsaSa  = [];
96 VsvSa  = [];
97 VrvSa  = [];
98 VpaSa  = [];
99
100 %Initialize empty vectors for solutions for flow
101 QpulSa  = [];
102 QmvalSa = [];
```

```
103 QavalSa = [];
104 QtvalSa = [];
105 QpvalSa = [];
106 QsysSa  = [];
107
108 options=odeset('RelTol',ODE_TOL, 'AbsTol',ODE_TOL);  %sets how accurate the ODE
    solver is
109
110 T  = data.Tinit;
111 k1 = 1; % index of first time step in first period
112 k2 = round(T/data.dt)+k1; %index of last time step in first period
113 Ts = length(data.td(k1:k2));
114 for i = 2:20 % go through loop NC times
115     clear Vpv Vlv Vsa Vsv Vrv Vpa
116     clear Ppv Plv Psa Psv Prv Ppa Elv Erv
117     clear Qpul Qmval Qaval Qtval Qpval Qsys
118
119     tdc = data.td(k1:k2); %current period
120
121     sol = ode15s(@modelBasic,tdc,Init,options,pars, parsG, tdc(1)); %the ODE
    solver calls modelBasic and enters in all the following values
122     sols= deval(sol,tdc); %takes the solutions at each time step in the period
123
124     %assigns each row as a temporary vector to store the solutions
125     %for the current time period
126     Vpv  = sols(1,:)';
127     Vlv  = sols(2,:)';
128     Vsa  = sols(3,:)';
129     Vsv  = sols(4,:)';
130     Vrv  = sols(5,:)';
131     Vpa  = sols(6,:)';
132
133     Ppv = (Vpv-VpvU)/Cpv;
134     Psa = (Vsa-VsaU)/Csa;
135     Psv = (Vsv-VsvU)/Csv;
136     Ppa = (Vpa-VpaU)/Cpa;
137
138     %Determines the elasticity of the left ventricle at each period timestep
139     for j = 1:length(tdc)
140         Elv(j) = Elastance(tdc(j)-tdc(1),ElvM, Elvm, TC, TR);
141
142         %Pressure of the left ventricle
143         Plv(j) = Elv(j).*(Vlv(j) - VlvU);
144
145         % flow into LH (mitral valve)
146         if Ppv(j) > Plv(j)
147             Qmval(j) = (Ppv(j)-Plv(j))/Rmval;
148         else
149             Qmval(j) = 0;
150
151         end
152         % flow out of LH (aortic valve)
153         if Plv(j) > Psa(j)
154             Qaval(j) = (Plv(j)-Psa(j))/Raval;
155         else
```

```
            Qaval(j) = 0;
        end
    end

    %Determines the elasticity of the right ventricle at each period timestep
    for j = 1:length(tdc)
        Erv(j) = Elastance(tdc(j)-tdc(1),ErvM, Ervm, TC, TR);

        %Pressure of the right ventricle
        Prv(j) = Erv(j).*(Vrv(j) - VrvU);

       % flow into RH (tricuspid valve)
        if Psv(j) > Prv(j)
            Qtval(j) = (Psv(j)-Prv(j))/Rtval;
        else
            Qtval(j) = 0;
        end
        % flow out of RH (pulmonary valve)
        if Prv(j) > Ppa(j)
            Qpval(j) = (Prv(j)-Ppa(j))/Rpval;
        else
            Qpval(j) = 0;
        end
    end

    %Flows defined by Ohm's Law
    Qpul = (Ppa - Ppv)/Rpul;
    Qsys = (Psa - Psv)/Rsys;

    % Adds to pressure solution vectors every iteration of the loop
    PpvSa  = [PpvSa  Ppv(1:end-1)'];
    PlvSa  = [PlvSa  Plv(1:end-1)'];
    PsaSa  = [PsaSa  Psa(1:end-1)'];
    PsvSa  = [PsvSa  Psv(1:end-1)'];
    PrvSa  = [PrvSa  Prv(1:end-1)'];
    PpaSa =  [PpaSa  Ppa(1:end-1)'];

    % Adds to volume solution vectors every iteration of the loop
    VpvSa  = [VpvSa  Vpv(1:end-1)'];
    VlvSa  = [VlvSa  Vlv(1:end-1)'];
    VsaSa  = [VsaSa  Vsa(1:end-1)'];
    VsvSa  = [VsvSa  Vsv(1:end-1)'];
    VrvSa  = [VrvSa  Vrv(1:end-1)'];
    VpaSa =  [VpaSa  Vpa(1:end-1)'];

    % Adds to flow solution vectors every iteration of the loop
    QpulSa  = [QpulSa   Qpul(1:end-1)'];
    QmvalSa = [QmvalSa  Qmval(1:end-1)'];
    QavalSa = [QavalSa  Qaval(1:end-1)'];
    QtvalSa = [QtvalSa  Qtval(1:end-1)'];
    QpvalSa = [QpvalSa  Qpval(1:end-1)'];
    QsysSa  = [QsysSa   Qsys(1:end-1)'];

    Init = [Vpv(end) Vlv(end) Vsa(end) Vsv(end) Vrv(end) Vpa(end)];

```

```
    if i < 20
      T  = (data.per(i+1)-data.per(i));

      k1 = k2; %sets last index of this loop as first index for next loop
      k2 = round(T/data.dt)+k1;
      if k2 > length(data.td)
          k2 = length(data.td);
      end
    end
end

%After for loop is done, saves last value for each of these vectors
PpvSa  = [PpvSa  Ppv(end)];
PlvSa  = [PlvSa  Plv(end)];
PsaSa  = [PsaSa  Psa(end)];
PsvSa  = [PsvSa  Psv(end)];
PrvSa  = [PrvSa  Prv(end)];
PpaSa =  [PpaSa  Ppa(end)];

VpvSa  = [VpvSa  Vpv(end)];
VlvSa  = [VlvSa  Vlv(end)];
VsaSa  = [VsaSa  Vsa(end)];
VsvSa  = [VsvSa  Vsv(end)];
VrvSa  = [VrvSa  Vrv(end)];
VpaSa =  [VpaSa  Vpa(end)];

QpulSa  = [QpulSa   Qpul(end)];
QmvalSa = [QmvalSa  Qmval(end)];
QavalSa = [QavalSa  Qaval(end)];
QtvalSa = [QtvalSa  Qtval(end)];
QpvalSa = [QpvalSa  Qpval(end)];
QsysSa  = [QsysSa   Qsys(end)];

figure(20);clf;
h=plot(data.td(Ts:end),data.pd(Ts:end),'r',data.td(Ts:end),PpaSa(Ts:end),'b');
set(gca,'fontsize',24);
set(h,'linewidth',2);
xlabel('time (s)');
ylabel('Ppa (mmHg)');
legend('data','model');
xlim([data.td(Ts) data.td(end)]);
grid on;
if opt == 1
  spf2 = strcat('modelPau_opt',num2str(datn),'.png');
else
  spf2 = strcat('modelPau_nom',num2str(datn),'.png');
end
print(spf2,'-dpng');

N = length(data.td);
N = N-min(6*length(tdc),length(data.td))+1;
figure(30);clf;
h=plot(data.td(N:end),PpaSa(N:end),data.td(N:end),PsaSa(N:end),...
       data.td(N:end),PsvSa(N:end),data.td(N:end),PpvSa(N:end),...
       data.td(N:end),PlvSa(N:end), data.td(N:end), PrvSa(N:end));
```

```
set(gca,'fontsize',24);
set(h,'linewidth',2);
xlabel('time (s)');
ylabel('pressure (mmHg)');
xlim([data.td(N) data.td(end)]);
legend('Ppa','Psa','Psv','Ppv','Plv', 'Prv');
grid on;
if opt == 1
  spf3 = strcat('modelAllPres_opt',num2str(datn),'.png');
else
  spf3 = strcat('modelAllPres_nom',num2str(datn),'.png');
end
print(spf3,'-dpng');

ss = strcat('Res',num2str(datn),'.mat');
save(ss,'data','PpaSa');

```